

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Инженерия Искусственного Интеллекта

Лабораторная работа № 1

Дисциплина: Фронтенд разработка

“Верстка сайта средствами HTML, CSS и JavaScript”

Выполнил:

Фиолетов Эдуард Анджеевич

Группа J3213

Проверил:

Добряков Д. Ю.

Санкт-Петербург

2026 г.

Содержание

1. Цель работы	3
2. Введение и обоснование выбранного стека	3
3. Задание	3
4. Используемые технологии	3
5. Структура интерфейса	4
6. Основные страницы	5
6.1. Страница входа	5
6.2. Выдача доступа пользователю	5
6.3. Личный кабинет и обзор	5
6.4. Страница заказов	5
6.5. Поиск и фильтрация	6
6.6. Страница доставки	6
7. Стилизация	6
8. Интерактивные элементы	7
9. Проверка результата	7
10. Вывод	7

1. Цель работы

Цель лабораторной работы - разработать интерфейс веб-приложения по выбранному варианту, подготовить основные пользовательские страницы и реализовать интерактивные элементы клиентской части.

В качестве варианта выбран сервис мониторинга заказов маркетплейсов и управления доставкой **Marketplace Alarm Bot 2**. Приложение предназначено для менеджеров, которые работают с заказами Wildberries, Ozon, Яндекс Маркета и Авито, распределяют заказы по городам, контролируют даты доставки и состояние интеграций.

2. Введение и обоснование выбранного стека

Для проекта Marketplace Alarm Bot 2 использован стек React, TypeScript, Vite, Tailwind CSS и Go API. Приложение работает как внутренний инструмент с большим количеством экранов, ролей и состояний, поэтому интерфейс собран как компонентная SPA, а не как набор отдельных статических страниц.

React выбран для компонентной разработки интерфейса: страницы, таблицы, карточки заказов, модальные окна и навигация вынесены в переиспользуемые компоненты. TypeScript используется для описания DTO и снижения риска ошибок при работе с данными заказов. Tailwind CSS и UI-компоненты задают единообразную верстку и дают гибкий контроль над плотной операционной панелью. Vite используется для сборки и локальной разработки, а Go-сервер обслуживает JSON API и собранную SPA-статiku.

3. Задание

По заданию необходимо выполнить верстку сайта, выбранного из набора вариантов курса или согласованного собственного варианта. Сайт должен содержать базовые страницы:

- вход в систему;
- регистрацию или выдачу доступа пользователю;
- личный кабинет пользователя;
- страницу поиска и фильтрации;
- дополнительные страницы, необходимые для выбранной предметной области;
- интерактивные элементы на JavaScript.

Для выбранного варианта набор страниц построен вокруг реальной предметной области маркетплейсов: обзор, списки заказов по площадкам, доставка по городам, архив, статистика, справочник товаров, склады, состояние сервиса и административные настройки.

4. Используемые технологии

В работе использованы HTML, CSS и JavaScript/TypeScript. Интерфейс собран как компонентное приложение на React, стилизация выполнена через Tailwind CSS

и набор переиспользуемых UI-компонентов. Проект собирается Vite и размещается в каталоге:

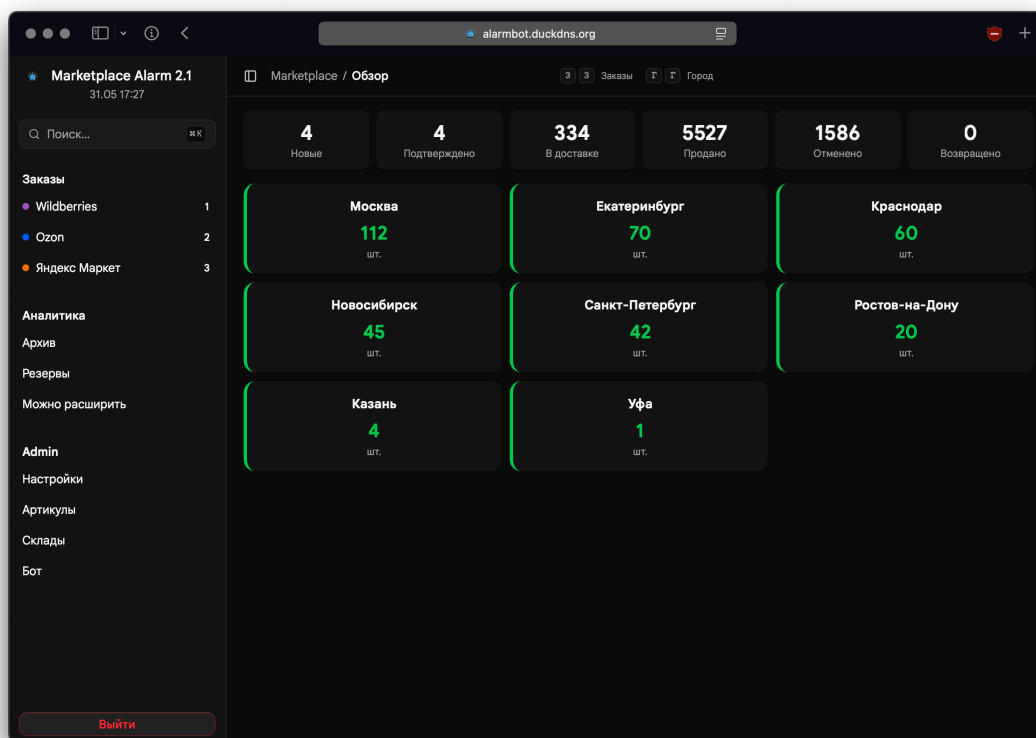
- ./web/frontend - клиентская часть веб-интерфейса
- ./web/frontend/src/pages - страницы приложения
- ./web/frontend/src/components - переиспользуемые компоненты интерфейса
- ./web/frontend/src/index.css - глобальные стили и CSS-переменные

5. Структура интерфейса

Главная структура приложения задается файлом `src/App.tsx`. В нем подключены страницы входа, обзора, заказов, архива, доставки, статистики, товаров, состояния сервиса, складов и настроек администратора. Страницы подключаются лениво, что уменьшает стартовый объем загружаемого кода.

```
const LoginPage = lazy(() => import('./pages/LoginPage'));
const IndexPage = lazy(() => import('./pages/IndexPage'));
const OrdersPage = lazy(() => import('./pages/OrdersPage'));
const DeliveryPage = lazy(() => import('./pages/DeliveryPage'));
const ProductsPage = lazy(() => import('./pages/ProductsPage'));
const WarehousesPage = lazy(() => import('./pages/WarehousesPage'));
```

Навигационный каркас реализован в `src/components/AppLayout.tsx`. Он содержит боковое меню, верхнюю панель, хлебные крошки, переключатели инструментов доставки и командную палитру. Каркас дает быстрые переходы между рабочими разделами и сохраняет единый визуальный стиль.



Главная страница приложения: боковая навигация, верхняя панель, карточки статистики и список городов доставки.

6. Основные страницы

6.1. Страница входа

Страница входа находится в `src/pages/LoginPage.tsx`. На ней реализована форма с полем ключа доступа и кнопкой отправки. Для защиты от случайной отправки пустой формы используется атрибут `required`, поле получает фокус автоматически, а при ошибке показывается сообщение.

```
<Input
  type="password"
  value={key}
  onChange={(e) => setKey(e.target.value)}
  placeholder="Key"
  autoFocus
  required
/>
```

6.2. Выдача доступа пользователю

Для выбранной предметной области регистрация пользователя реализована как выдача ключа доступа. На серверной стороне предусмотрены административные методы `/api/admin/keys`, которые позволяют создать и удалить ключ, а также назначить ему уровень доступа. Такой вариант соответствует внутреннему сервису, где пользователей добавляет администратор, а самостоятельная публичная регистрация не требуется.

Уровни доступа:

- `full` - полный доступ к настройкам и административным действиям;
- `manager` - доступ менеджера к заказам, доставке, складам и статистике;
- `city` - доступ только к доставке конкретного города.

6.3. Личный кабинет и обзор

Главная страница `src/pages/IndexPage.tsx` выполняет роль личного кабинета менеджера. На ней выводятся агрегированные показатели: количество активных заказов, статистические карточки и список городов с количеством товаров к доставке. Карточки городов ведут на соответствующие страницы доставки.

```
{data.cities.map((c) => (
  <Link to={` /delivery/${encodeURIComponent(c.city)} `}>
    <div>{c.city}</div>
    <div>{c.qty}</div>
  </Link>
))}
```

6.4. Страница заказов

Страница `src/pages/OrdersPage.tsx` отображает заказы выбранного маркетплейса. Маркетплейс берется из параметра адреса `/orders/:marketplace`, после чего данные передаются в компонент таблицы `OrdersTable`.

Разделы заказов:

- /orders/wb - Wildberries;
- /orders/ozon - Ozon;
- /orders/yandex - Яндекс Маркет.

6.5. Поиск и фильтрация

Поиск реализован через командную палитру `src/components/CommandPalette.tsx`. Она поддерживает два режима: переход по страницам и поиск заказов. Для поиска заказа пользователь вводит имя клиента, телефон, город или внешний идентификатор. После выбора результата приложение открывает страницу доставки и подсвечивает нужный заказ.

```
function goOrder(order) {
  go(`/delivery/${encodeURIComponent(order.city)}?
order=${encodeURIComponent(order.externalId)}`);
}
```

6.6. Страница доставки

Страница `src/pages/DeliveryPage.tsx` является основным рабочим экраном. Она содержит временную шкалу, колонки маркетплейсов, панель подтвержденных заказов, drag-and-drop перенос карточек и карту доставок. Пользователь может выбрать дату, открыть карточку заказа, подтвердить доставку и посмотреть расположение заказов на карте.

В интерфейсе используются:

- сетка из колонок по маркетплейсам;
- карточки заказов с краткой информацией;
- липкая панель выбранной даты;
- drag-and-drop для переноса заказа в подтвержденные;
- модальное окно деталей заказа;
- плавающая карта заказов.

7. Стилизация

Глобальные стили находятся в `src/index.css`. В файле определены CSS-переменные для цветов, фона, границ, состояния элементов, цветов маркетплейсов и шрифтов. Для светлой и темной тем используются отдельные наборы переменных.

```
:root {
  --background: rgb(255, 255, 255);
  --foreground: rgb(10, 10, 10);
  --card: rgb(255, 255, 255);
  --mp-wb: #9b59b6;
  --mp-ozon: #005bff;
  --mp-yandex: #ff7300;
}
```

За счет этого одни и те же компоненты работают в разных темах без дублирования оформления на уровне отдельных страниц.

8. Интерактивные элементы

В работе реализованы следующие интерактивные элементы:

- форма входа с обработкой отправки;
- переходы по разделам через боковое меню;
- командная палитра для поиска и навигации;
- переключение периода доставки;
- включение и закрытие карты;
- drag-and-drop карточек заказов;
- модальное окно карточки заказа;
- редактирование складских меток и флагов;
- подсветка выбранного заказа из строки поиска.

Пример обработки входа:

```
const submit = async (e: FormEvent) => {  
  e.preventDefault();  
  const res = await fetch('/api/login', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ key })  
  });  
};
```

9. Проверка результата

Проверка выполнялась по пользовательским сценариям:

1. Открыть страницу входа и ввести ключ доступа.
2. Перейти на главную страницу и проверить отображение статистики.
3. Открыть список заказов маркетплейса.
4. Найти заказ через командную палитру.
5. Открыть страницу доставки города.
6. Перенести заказ на выбранную дату доставки.
7. Открыть карту и модальное окно деталей заказа.
8. Проверить, что элементы интерфейса не ломают сетку при разных объемах данных.

10. Вывод

В лабораторной работе разработан интерфейс веб-приложения Marketplace Alarm Bot 2. Реализованы страницы входа, обзора, заказов, поиска, доставки, складов и настроек. Интерфейс использует единый набор компонентов, адаптивную сетку, CSS-переменные и интерактивные элементы на стороне клиента. Реализация покрывает требования к верстке сайта по выбранному варианту и подготавливает основу для подключения внешнего API в следующей лабораторной работе.